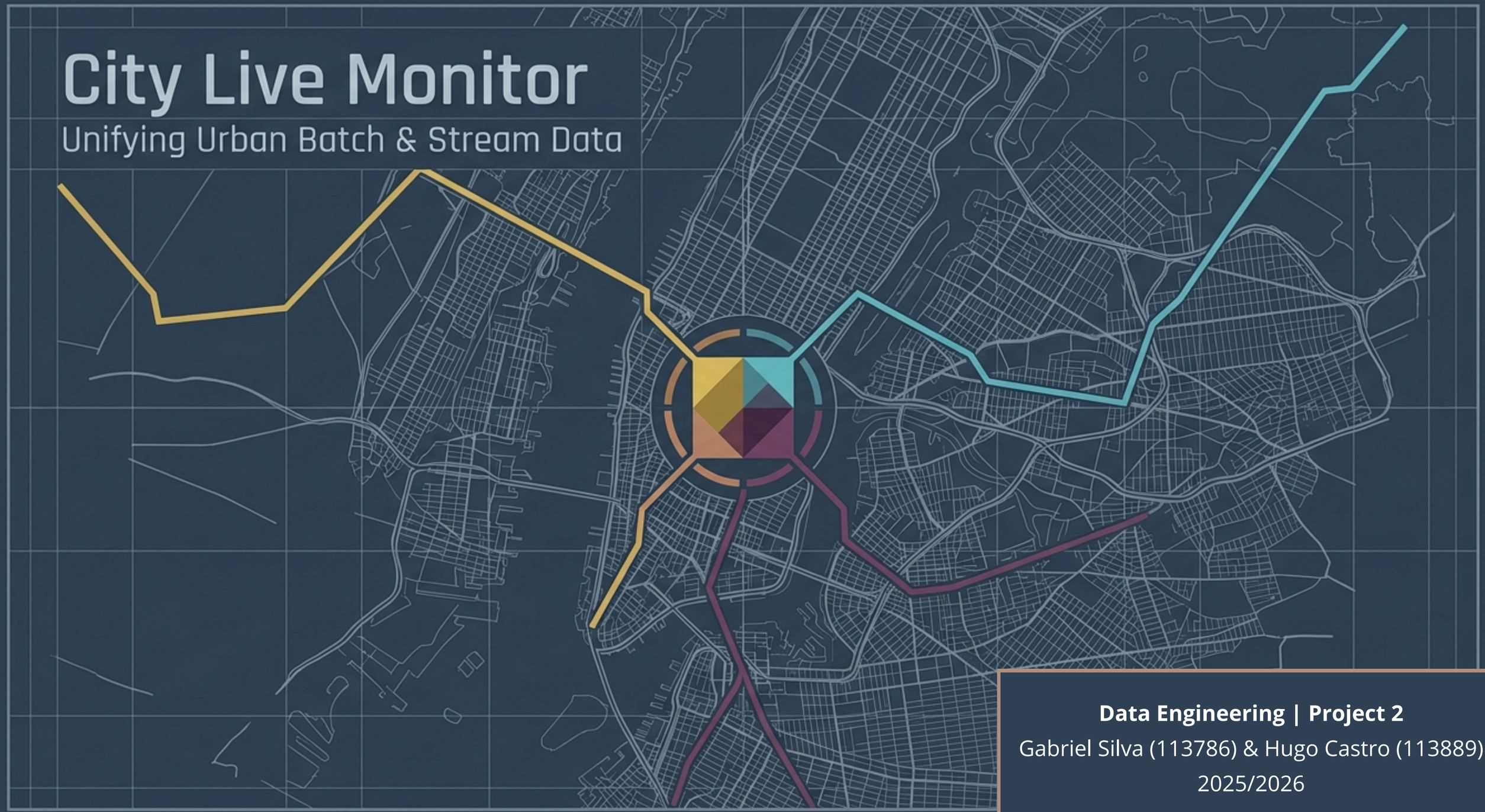


Data Engineering | Project 2

Gabriel Silva (113786) & Hugo Castro (113889)

2025/2026

Three Dimensions of NYC Urban Dynamics

TLC Taxi

- Batch Source
- 35 Parquet Files
- Jan 2023 - Nov 2025
- 123.9M raw rows

Captures dispatch, geospatial zones, and fares.

Permitted Events

(Real-time Periodic)

- Streaming Source (Socrata API)
- Polled every 30 mins
- 204K raw rows

Captures planned urban demand spikes.

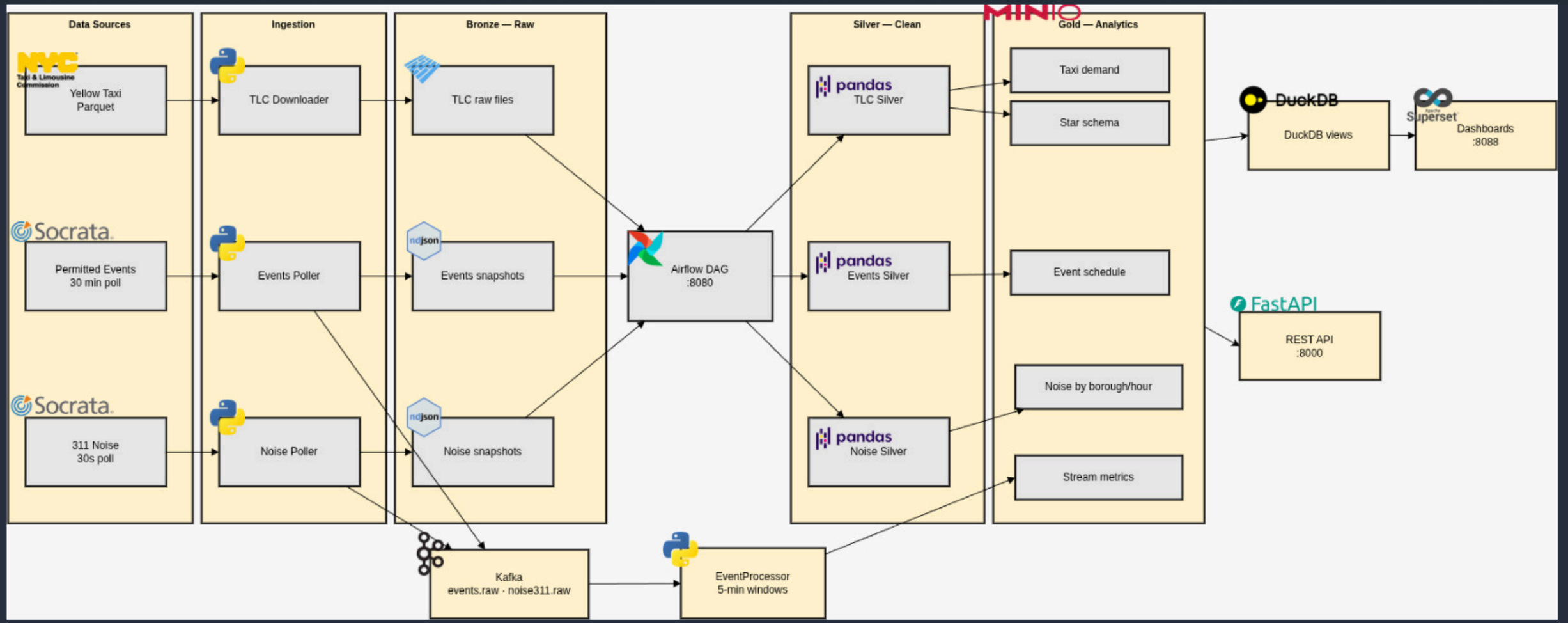
311 Noise

(Real-time/Event-based)

- Streaming Source (Socrata API)
- Polled every 30 seconds
- 100K raw rows

Captures real-time urban friction.

End-to-End Medallion Architecture



Robust Ingestion: Incremental Batch & Checkpointed Streaming

TLC Batch Strategy		Socrata Kafka Strategy	
	Atomic Writes (.tmp renames)		Incremental Checkpoints
	Parallel Threads with Exponential Backoff		Circuit Breakers (max 5 failures)
	Sidecar Metadata (.meta.json)		Dead-Letter Queues (DLQ) for malformed events

Bronze Layer Diagnostics: Profiling the Raw State

TLC Taxi



1.44% zero/negative trip distances.

0.85% negative fares.

Extreme outliers (>100 miles) detected.

Events



95.7% null rate in event_street_side.

High volume of cross-snapshot duplicates.

311 Noise

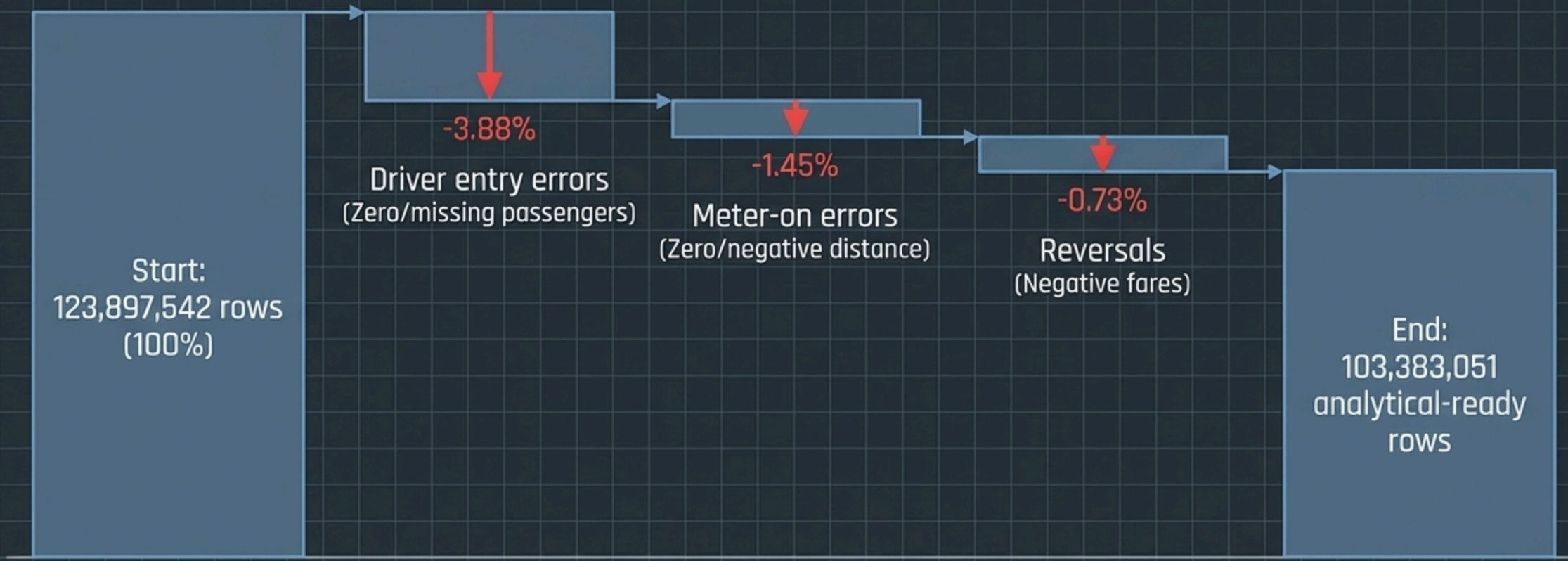


Null coordinate values present.

Timestamps lacking timezone awareness.

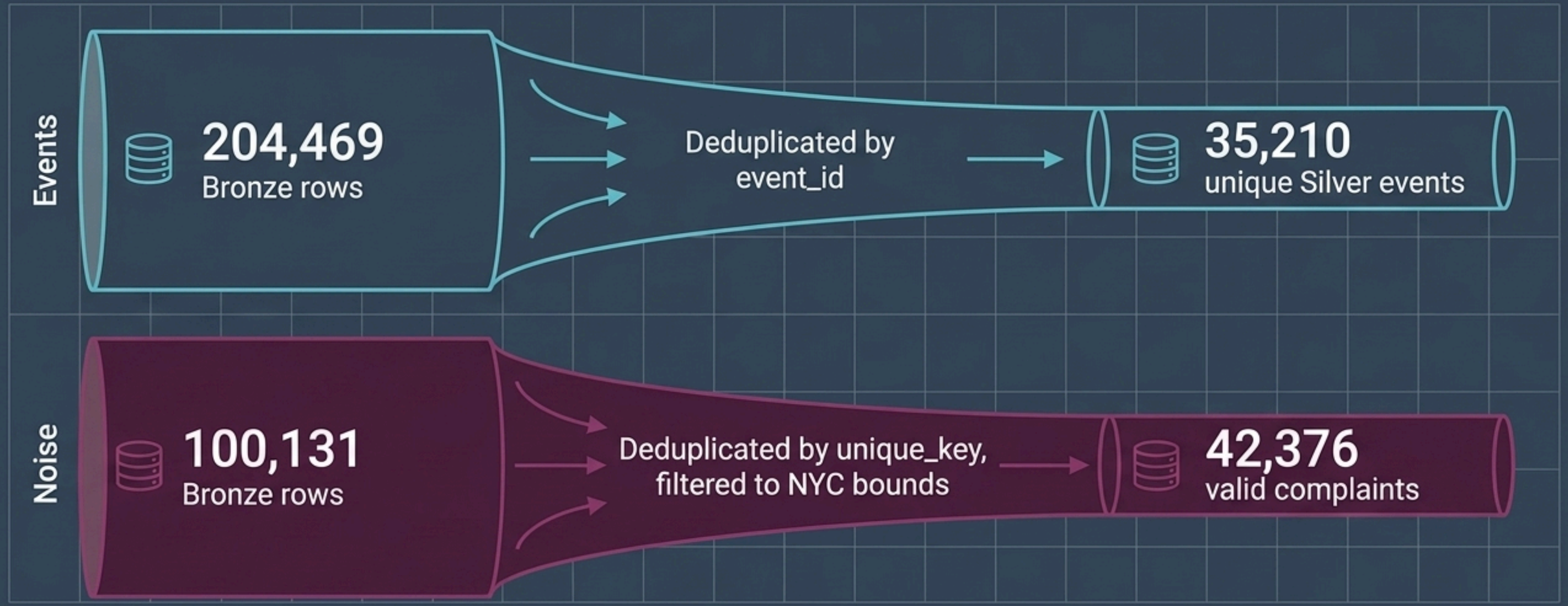
Key Takeaway: 0 Bronze Validation Issues post-ingestion. Implemented column-removal-only drift detection to resolve legacy schema drift.

Silver Layer: Filtering TLC Data to 83.4% Retention



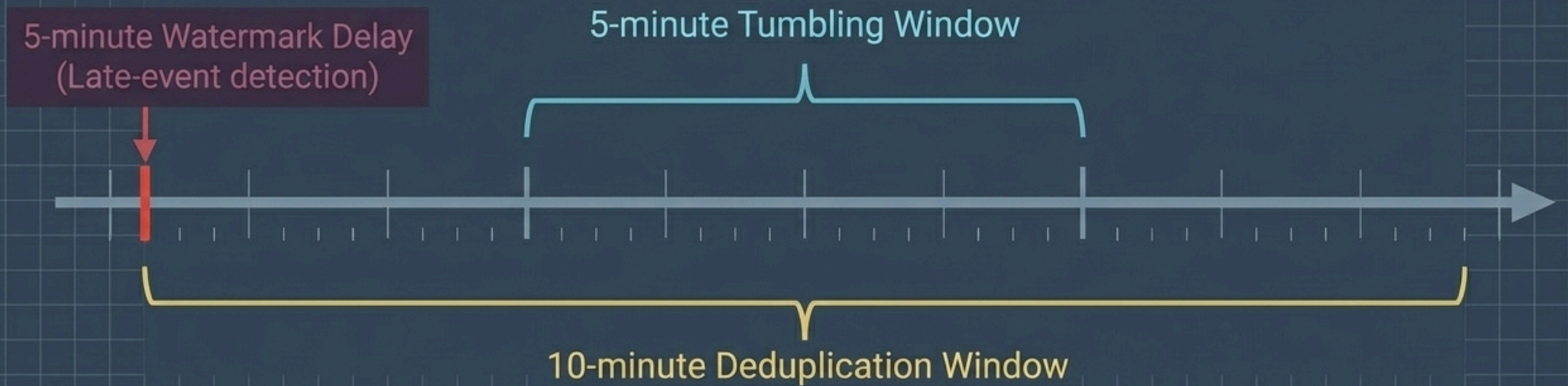
19 source columns preserved; 14 derived features engineered. Outliers are flagged, not filtered.

Silver Layer: Stream Deduplication & Normalization



High duplicate rate expected and handled: **Socrata endpoints return** it **full historical datasets on every poll.**

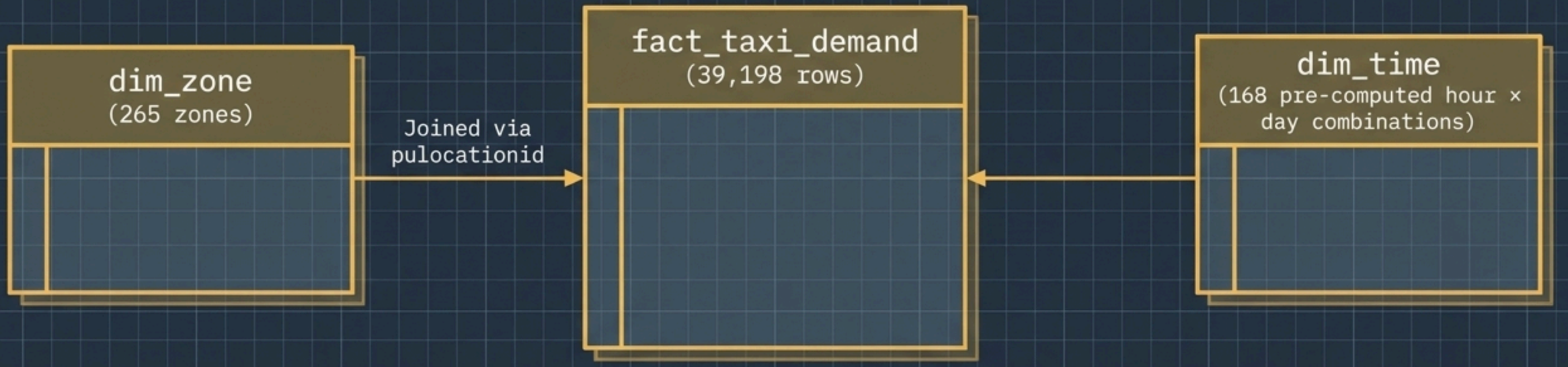
Handling Live Events via Windowed Aggregation



Outputs to the `stream_metrics_5m` Gold table:

- Event counts
- Late ratios
- Out-of-order counts

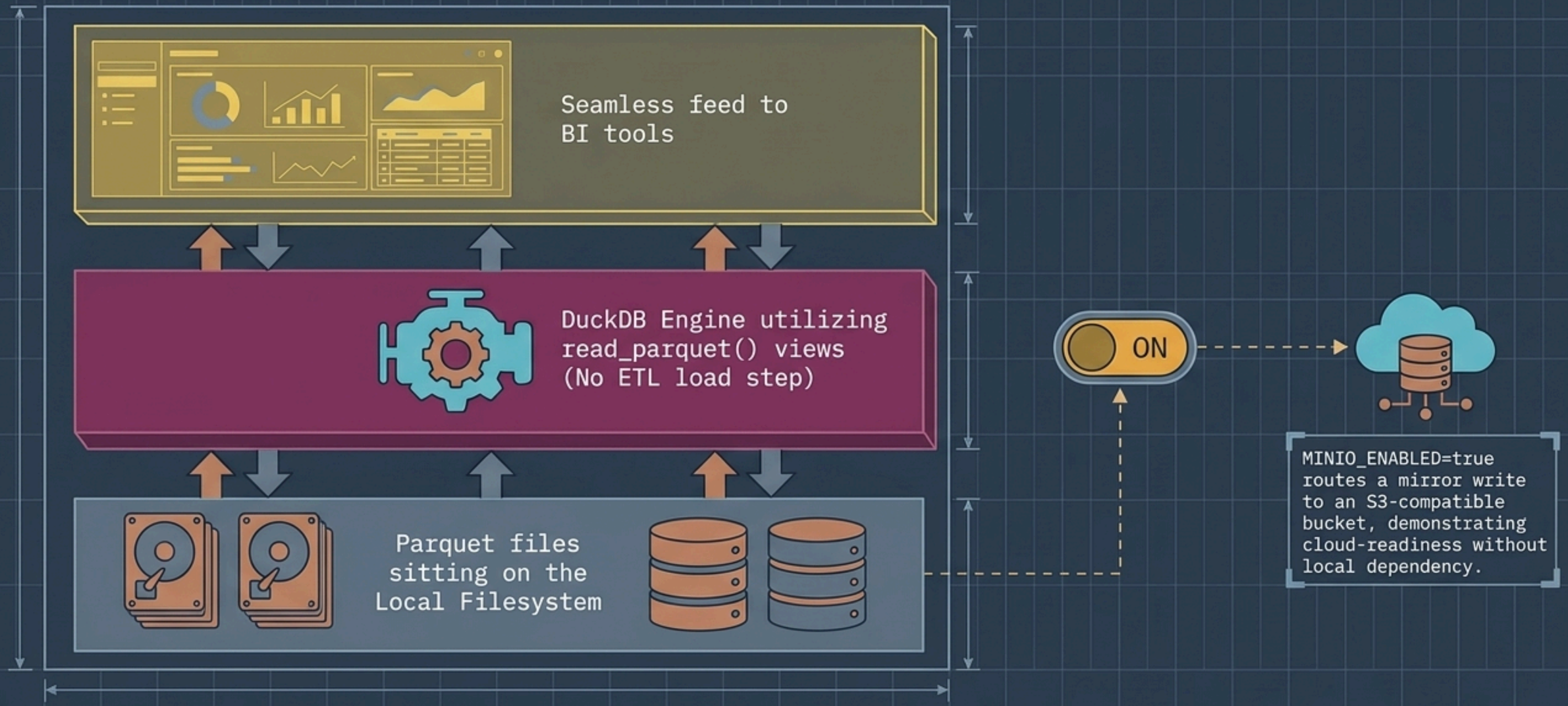
Gold Layer: The Analytical Data Model



Flat Table: event_schedule

Flat Table: noise_by_borough_hour

The Lakehouse Engine: Direct Querying via DuckDB



Exposing Data via the FastAPI Serving Layer

`/gold/taxi-demand`
Filters: zone, hour, weekend

`/gold/events`
Filters: nightlife flag, timeframe

`/gold/noise`
Filters: descriptor, borough

Response Envelope Structure:

```
{
  count: N,
  returned: M,
  rows: [...]
}
```

⚠ Strict HTTP 503 error handling for unbuilt tables.

Apache Superset Integration

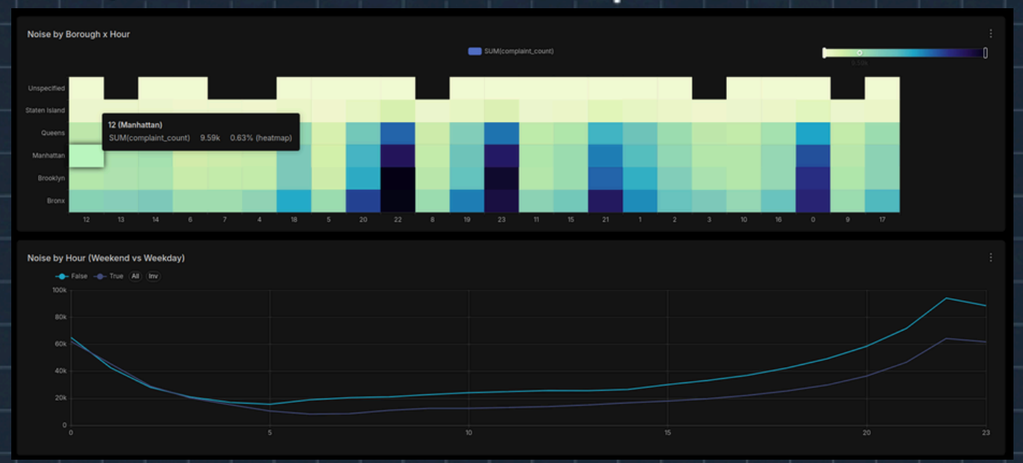
Q1: Late-Night Taxi



Q2: Tonight's Events



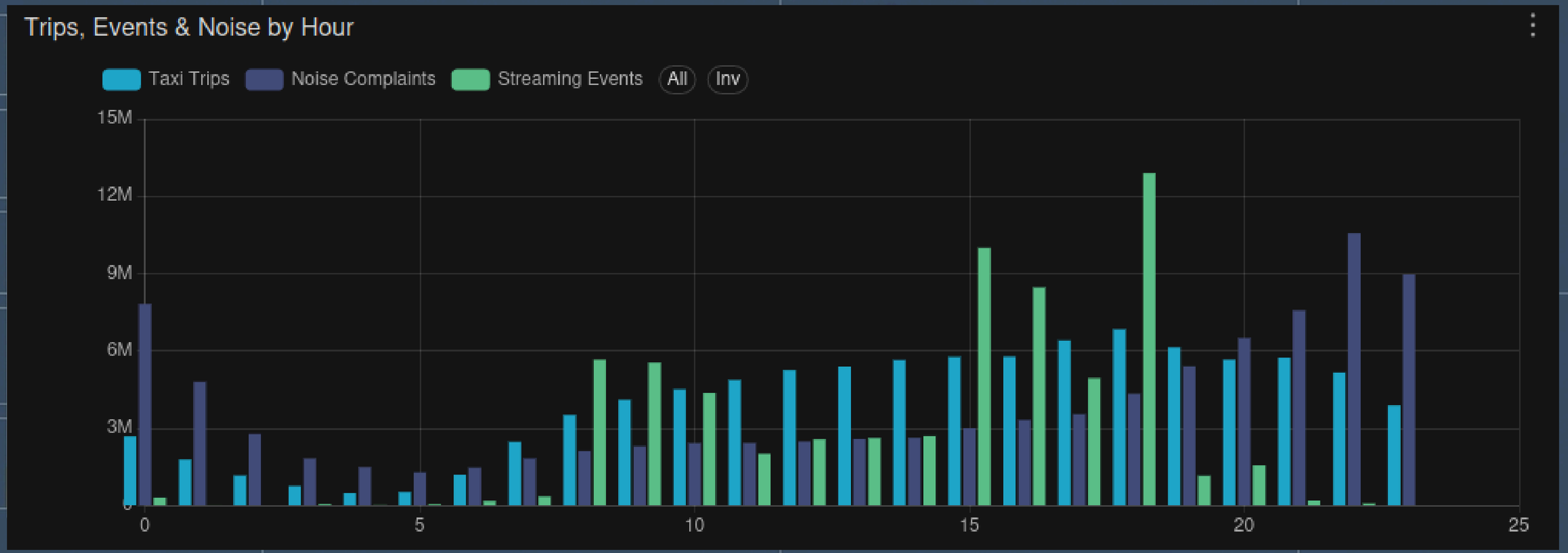
Q3: Live Noise Complaints



Q4: Batch + Streaming Combined Dashboard



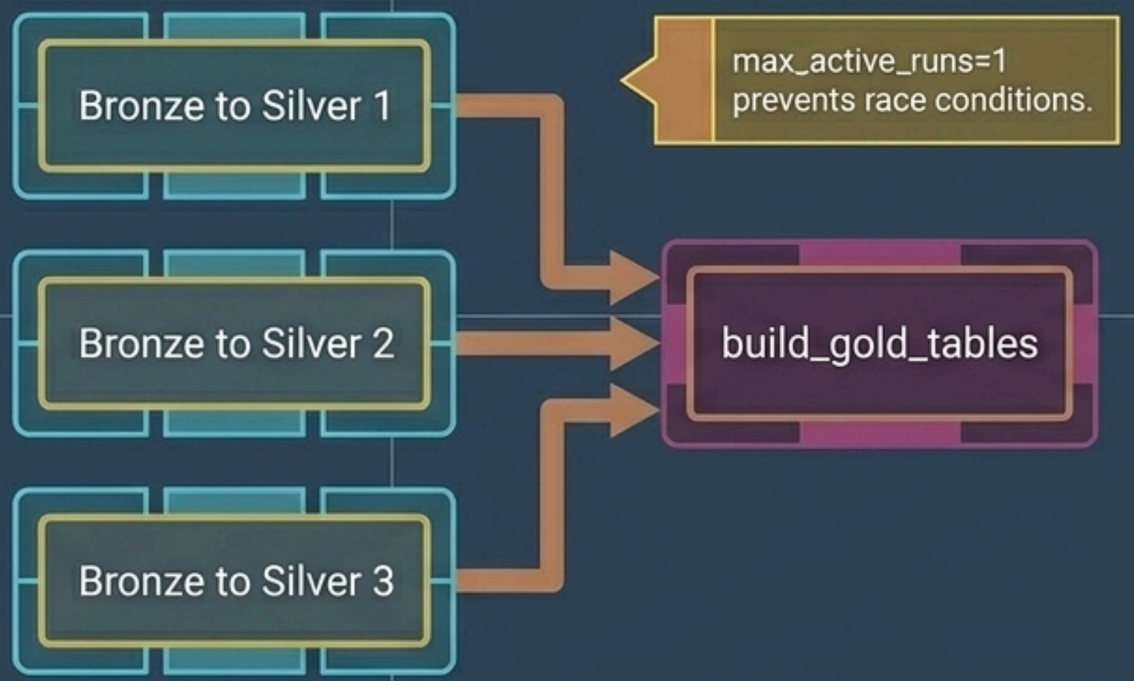
Synthesis: Aligning Historical Demand with Live Urban Friction



By combining historical **time-of-week** patterns with **real-time** event schedules, the platform contextualizes live urban friction.

Automated Orchestration & Final Validation

The Engine



The Results

124M → 103M
Rows Cleaned

0 Bronze Validation Issues

7 High-Performance Gold Tables Built

A fully idempotent, hybrid batch/streaming platform executed entirely on open-source infrastructure.

FUTURE WORK



1) **Operational Monitoring:** Prometheus + Grafana for pipeline health (metrics like Kafka lag and circuit breaker state).



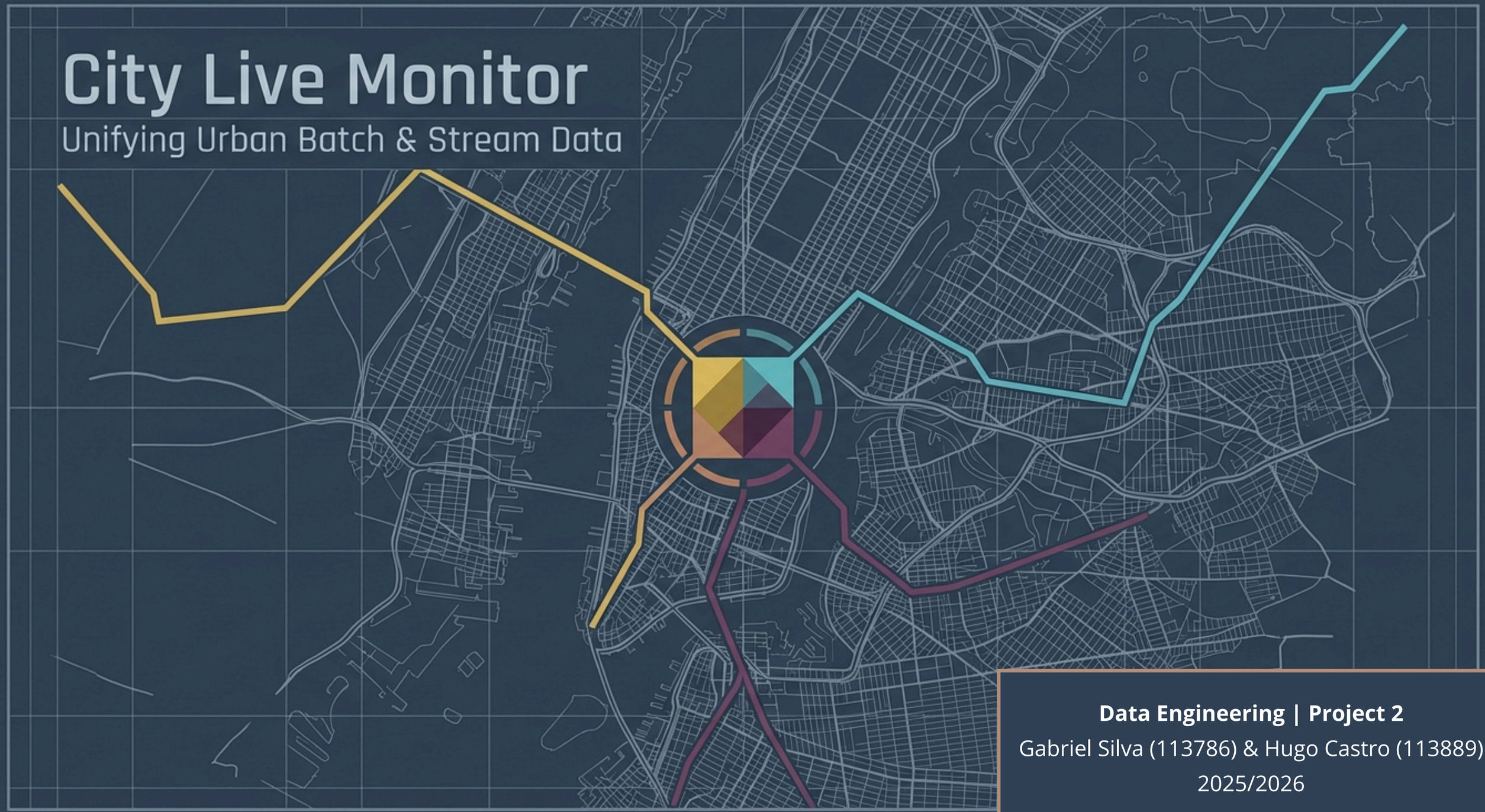
2) **Continuous Streaming:** Running Kafka runners as daemon processes for 24/7 operations.



3) **Delta Lake Integration:** Replacing plain Parquet for ACID transactions and time travel.



4) **Kafka Connect:** Replacing custom pollers with production-grade connectors.



Data Engineering | Project 2

Gabriel Silva (113786) & Hugo Castro (113889)

2025/2026